BéninConnect

Dossier technique fonctionnalités, architecture et configuration

Application communautaire de la diaspora béninoise : application mobile iOS et Android, API .NET en monolithe modulaire, console d'administration, site public et infrastructure de production.

Éditeur : HBA Tech et Trade · Auteur : Hector Adjakpa
Version 2.0 du document · 9 novembre 2026
Domaines : monbenin.com · api.monbenin.com

Sommaire

1 Présentation du projet
2 Guide fonctionnel de l'application
3 Vue d'ensemble de l'architecture
4 Structure du dépôt
5 Backend : monolithe modulaire
6 Modules métier et API
7 Communication entre modules
8 Sécurité
9 Temps réel et notifications
10 Médias et stockage
11 Application mobile
12 Console d'administration et site public
13 Configuration
14 Infrastructure et déploiement
15 CI/CD
16 Tests et qualité
17 Publication sur les stores
18 Points restants et évolutions
A Référence complète de l'API
B Modèle de données
C Flux détaillés
D Exploitation au quotidien
E Historique des évolutions
F Glossaire

Le projet en chiffres

9 modules backend	**~97** routes HTTP + 1 hub temps réel	**8** schémas PostgreSQL	**4** applications : mobile, API, console, site
67 tests backend (dont 2 S3)	**46** tests mobile (Jest)	**.NET 10** API	**Expo 58** React Native 0.88

> Ce document décrit l'état du projet au 9 novembre 2026 : code, configuration et infrastructure de production. Les secrets (clés, mots de passe, jetons) n'y figurent jamais : ils vivent uniquement dans le fichier `.env` du serveur, les secrets GitHub et les identifiants EAS.

1. Présentation du projet

BéninConnect réunit les Béninoises et les Béninois, où qu'ils vivent (Paris, Lyon, Bruxelles, Montréal, Cotonou…). L'application permet de retrouver les membres de sa ville, de partager des nouvelles, de découvrir les événements de la communauté et d'échanger en privé.

Fonctionnalités

Domaine	Fonctionnalités
Compte	Inscription avec code de vérification par email (Resend), connexion, mot de passe oublié, profil (photo, couverture, profession, ville, centres d'intérêt), confidentialité, **blocage d'autres membres**, **suppression du compte** dans l'application
Communautés	Communautés publiques ou privées (demandes d'adhésion), catégories, membres, création
Fil d'actualité	Publications texte, **photo ou vidéo** (60 s), visibilité (publique, communauté, moi seul), j'aime, commentaires, réactions emoji, partage natif, **modification et suppression** par l'auteur
Événements	Création, filtres (ville, catégorie, période), inscription, capacité
Messagerie	Conversations privées **en temps réel** (SignalR), réactions, accusés de lecture, **modification (15 min) et suppression** des messages, **pièces jointes** : photos, PDF, messages vocaux
Notifications	Centre de notifications, badge, **notifications push** iOS (APNs) et Android (FCM) via Expo, catégories réglables
Sécurité de la communauté	Signalement des publications, commentaires, membres et communautés ; **blocage** d'un membre (contenus masqués et messages impossibles dans les deux sens) ; modération (masquage, suppression, suspension) dans la console

Composants livrés

Composant	Technologie	Rôle
Application mobile	Expo 58, React Native 0.88, TypeScript	Application iOS et Android des membres (`mobile-expo/`)
API	ASP.NET Core .NET 10, EF Core, PostgreSQL	Monolithe modulaire, REST + SignalR (`server/`)
Console d'administration	React 19, Vite, TanStack Query	Modération, membres, statistiques, audit (`admin-console/`)
Site public	HTML/CSS statique	Accueil, support, confidentialité, conditions, mentions légales, suppression de compte (`site/`)
Prototype historique	Avalonia (.NET 9)	Première maquette hors ligne (`src/` , `tests/`), exclue de la CI

2. Guide fonctionnel de l'application

Ce chapitre décrit l'application telle que la voit un membre : les parcours, chaque écran, ce qu'il permet et les règles qui s'y appliquent. Les noms de boutons sont ceux de l'interface (en français).

2.1 Premier lancement et compte

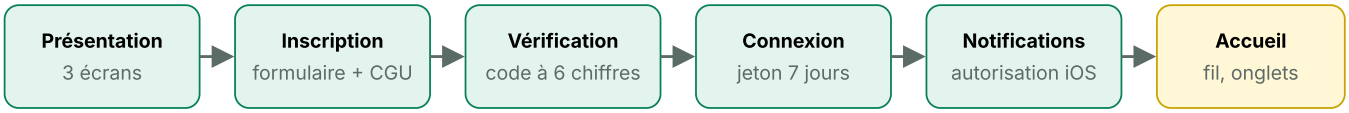


Figure 1 — Parcours d'un nouveau membre

Écran	Contenu et règles
Présentation `onboarding`	Trois écrans d'accueil (communauté, partage et événements, réseau mondial), affichés une seule fois.
Inscription `(auth)/register`	Prénom et nom (2 à 40 caractères), email, pays de résidence, ville, mot de passe (8 caractères minimum, lettres et chiffres) et confirmation. Case obligatoire d'acceptation des conditions d'utilisation et de la politique de confidentialité, avec liens vers `monbenin.com/conditions` et `/confidentialite`.
Vérification `(auth)/verify-email`	Code à 6 chiffres envoyé par email (Resend), renvoi possible. Tant que l'email n'est pas vérifié, la connexion est refusée. La première vérification déclenche un message de bienvenue (`MemberVerified`).
Connexion `(auth)/login`	Email et mot de passe, option « Rester connecté » (jeton conservé dans le trousseau via expo-secure-store). Jeton valable 7 jours ; un compte suspendu est refusé immédiatement.
Mot de passe oublié `(auth)/forgot-password`	Code de réinitialisation par email, valable 30 minutes, 5 essais au maximum, puis saisie du nouveau mot de passe.

2.2 Navigation principale

Une barre d'onglets en verre dépoli donne accès à cinq zones : **Accueil**, **Communautés**, le bouton central **+** (Publier), **Messages** et **Profil**. Le bandeau de l'Accueil propose aussi la **recherche** (loupe) et les **notifications** (cloche avec pastille du nombre de non-lues). L'onglet Messages affiche le nombre de messages non lus.

2.3 Écrans et fonctionnalités

Écran	Ce que le membre peut faire
Accueil `(tabs)/home`	Communautés suggérées, événements à venir (« Voir tout » ouvre la liste des événements), fil d'actualité paginé avec rafraîchissement par glissement. Chaque publication : auteur, date, mention « modifié », communauté, visibilité, texte, photo ou vidéo, j'aime, commentaires, partage.
Publier `post/new`	Texte (2 000 caractères), une photo (galerie ou appareil, 10 Mo) *ou* une vidéo (60 s, 60 Mo, compressée en 720p sur iPhone, vignette générée), visibilité : *Public*, *Membres de la communauté* ou *Moi seulement*, communauté facultative. Barre de progression pendant l'envoi d'une vidéo. Le même écran sert à **modifier** une publication.
Détail d'une publication `post/[id]`	Publication complète, commentaires (1 000 caractères), réactions emoji sur les commentaires (appui long), lien « Signaler · Bloquer » sous les commentaires des autres membres.
Menu ⋯ d'une publication	Pour l'auteur : *Modifier*, *Supprimer* (avec confirmation). Pour les autres : *Signaler la publication*, *Bloquer [nom]*.
Communautés `(tabs)/communities`	Recherche, filtre par catégorie, « Mes communautés » puis les autres ; création d'une communauté (`community/new` : nom, description, catégorie, pays, ville, publique ou privée, image de couverture parmi une sélection).
Communauté `community/[id]`	Couverture, description, membres, publications et événements de la communauté ; *Rejoindre* (immédiat si publique, demande en attente si privée), *Quitter*, annuler sa demande, publier dans la communauté, signaler la communauté.
Événements `events/`	Liste filtrable (recherche, catégorie, ville, période : cette semaine, ce mois, plus tard), détail (date, lieu, organisateur, capacité, participants), *Participer* / *Se désinscrire*, création d'un événement (`events/new`), rattachement possible à une communauté. Inscription refusée quand l'événement est complet.
Messages `(tabs)/messages`	Liste des discussions triée par dernier message, aperçu (texte, « Photo », « Message vocal », « Document »), compteur de non-lus ; nouvelle discussion (`conversation/new`) en cherchant un membre.
Discussion `conversation/[id]`	Messages en temps réel, séparateurs par jour, accusés de lecture, réactions (appui long), *Modifier* (15 minutes, texte seul) et *Supprimer* ses messages. Trombone : photo de la galerie, prise de vue, document PDF (20 Mo). Micro : message vocal (5 minutes au plus), lu dans la bulle avec barre de progression, même en mode silencieux. Menu ⋮ : voir le profil, signaler, bloquer ou débloquer.
Recherche `search`	Recherche globale avec portée : Tout, Membres, Publications, Communautés, Événements, et suggestions (Paris, Montréal, Étudiants…).
Notifications `notifications`	Centre de notifications par catégorie, lecture individuelle ou « Tout marquer comme lu » ; toucher une notification ouvre l'élément concerné.
Profil `(tabs)/profile`, `user/[id]`	Couverture, photo, profession, ville, origine, bio, centres d'intérêt (6 au plus), statistiques, communautés et publications. Sur son profil : changer photo et couverture, *Modifier le profil* (`profile/edit`), Paramètres. Sur le profil d'un autre : *Message*, menu ⋯ (signaler, bloquer, débloquer).
Paramètres `settings`	Apparence (clair, sombre, système), confidentialité (profil privé, afficher sa ville, autoriser les messages, **Utilisateurs bloqués**), catégories de notifications (sociales, communautés, messages, événements — transmises au serveur), à propos, déconnexion, **Supprimer mon compte**.
Utilisateurs bloqués `account/blocked`	Liste des membres bloqués avec bouton *Débloquer*.
Supprimer mon compte `account/delete`	Explication des conséquences, confirmation par mot de passe, suppression définitive puis retour à l'écran de connexion.

2.4 Règles de confidentialité et de sécurité de la communauté

Règle	Comportement
Visibilité d'une publication	*Public* : tous les membres. *Membres de la communauté* : membres de la communauté choisie. *Moi seulement* : l'auteur seul. Une publication masquée par la modération n'est plus visible de personne.
Communauté privée	L'adhésion passe par une demande validée ; les publications réservées aux membres restent invisibles en attendant.
Signalement	Publications, commentaires, membres et communautés ; motifs : spam, harcèlement, contenu inapproprié, fausse information, autre (détails 500 caractères). Un même contenu ne peut être signalé deux fois par la même personne ; on ne peut pas signaler son propre contenu. Le signalement arrive dans la console de modération.
Blocage	Dans les deux sens : les publications et commentaires de l'autre disparaissent (fil, profils, recherche, détail) ; aucun des deux ne peut plus écrire à l'autre ni ouvrir de discussion ; la discussion existante reste lisible avec un bandeau « Vous avez bloqué… » ou « Vous ne pouvez plus répondre ». La personne bloquée n'est pas prévenue. Déblocage depuis le profil, la discussion ou les Paramètres.
Suspension	Décidée par la modération avec un motif : connexion refusée et jetons existants invalidés immédiatement.
Suppression de compte	Profil, publications, commentaires, j'aime, réactions, messages, adhésions, communautés créées, inscriptions aux événements, notifications, appareils push, signalements émis, blocages, photos et pièces jointes : tout est effacé (événement `MemberDeleted` traité par chaque module). Également demandable depuis `monbenin.com/supprimer-compte/` .
Pièces jointes	Stockées hors de l'espace public ; seuls les deux participants de la discussion peuvent les télécharger.

2.5 Notifications

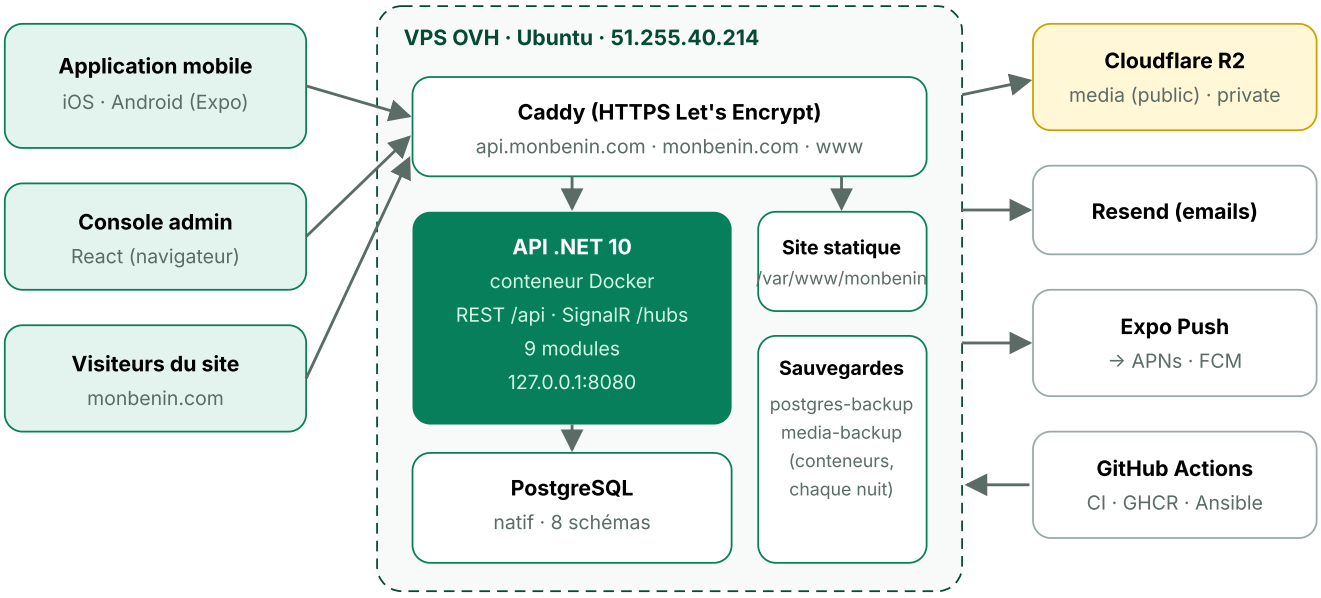
Événement	Catégorie	Destinataire et ouverture
J'aime sur une publication	Sociale	Auteur de la publication → publication
Commentaire	Sociale	Auteur de la publication → publication
Réaction à un commentaire	Sociale	Auteur du commentaire → publication
Adhésion à une communauté	Communautés	Le nouveau membre (message de bienvenue) → communauté
Nouveau message	Messages	Destinataire → discussion
Email vérifié	Système	Le membre (bienvenue dans BéninConnect)
Notification générique (`NotificationRequested`)	Au choix	Point d'entrée prévu pour les autres modules (rappels d'événements…)

- Chaque notification est enregistrée dans le centre de notifications et, si la catégorie est active, envoyée en push avec le badge du nombre de non-lues.
- Application ouverte : bannière et son, **sauf** pour la discussion actuellement affichée (le message arrive déjà en temps réel).
- Toucher une notification ouvre l'écran concerné ; depuis une autre discussion, l'écran est remplacé (pas d'empilement), et une même notification n'ouvre jamais deux écrans.

2.6 Limites et paramètres métier

Élément	Limite	Élément	Limite
Publication	2 000 caractères	Photo	10 Mo (JPEG, PNG, WebP, HEIC)
Commentaire	1 000 caractères	Vidéo	60 s, 60 Mo (MP4, MOV)
Message	2 000 caractères	Pièce jointe	20 Mo (photo, PDF, audio)
Modification d'un message	15 minutes	Message vocal	5 minutes (AAC .m4a)
Centres d'intérêt	6 au plus	Détails d'un signalement	500 caractères
Jeton membre	7 jours	Jeton administrateur	12 heures
Requêtes (production)	600 / min / client	Authentification	10 / min / client
Envois de fichiers	60 / heure / client	Code de réinitialisation	30 min, 5 essais

3. Vue d'ensemble de l'architecture



Flux principaux (les flèches vers l'extérieur partent de l'API)

1. Les clients appellent https://api.monbenin.com ; Caddy termine le TLS et relaie vers l'API (127.0.0.1:8080), WebSocket compris.
2. L'API lit et écrit dans PostgreSQL (un schéma et un DbContext par module) et range les fichiers dans R2 (ou le disque).
3. Les photos et vidéos publiques sont servies via l'URL R2 (redirection 301) ; les pièces jointes privées passent par l'API.
4. Chaque push sur main : CI (tests) → images Docker (GHCR) → déploiement Ansible sur le VPS.

Figure 2 — Architecture de production

Choix structurants

- **Monolithe modulaire** : un seul déploiement, mais des modules isolés (Clean Architecture), chacun avec son schéma de base, ses contrats et ses événements. Un module peut être extrait en microservice plus tard sans réécriture.
- **Serveur unique, simple à opérer** : Caddy installé sur l'hôte (HTTPS automatique), API en conteneur, PostgreSQL natif, stockage objet externe.
- **Application mobile unique** pour iOS et Android (Expo), avec une couche de données interchangeable (API réelle ou démonstration hors ligne).
- **Tout est automatisé** : tests, images, déploiement, configuration Caddy et site public par la CI/CD.

4. Structure du dépôt

```
BeninConnect/
├── server/                          # Backend
│   ├── src/BeninConnect.Api/          # API (projet unique)
│   │   ├── Program.cs                 # composition, pipeline, --migrate
│   │   ├── Host/                      # Production.cs, DatabaseStartup, modules, démo
│   │   ├── Shared/                    # noyau commun : Domain, Application, Infrastructure, Api
│   │   ├── Modules/                   # 9 modules (voir §5)
│   │   ├── appsettings*.json · Dockerfile
│   ├── tests/BeninConnect.Api.Tests/  # intégration (PostgreSQL/MinIO), architecture, domaine
│   ├── docker-compose.yml             # développement local
│   ├── docker-compose.prod.yml        # production : api, backup, media-backup
│   ├── Caddyfile                      # api.monbenin.com + monbenin.com
│   ├── .env.example                   # modèle de configuration (sans secrets)
│   └── tools/media-to-s3.sh           # copie disque → R2
├── mobile-expo/                     # Application mobile
│   ├── src/app/                       # écrans (expo-router) : (auth), (app), onboarding
│   ├── src/components/                # UI, cartes, pièces jointes, vidéo, réactions
│   ├── src/core/                      # config, session, push, thème, photos, images…
│   ├── src/data/                      # contrat DataSource, client API, mock
│   ├── src/features/                  # requêtes, temps réel, signalement, enregistrement vocal
│   ├── plugins/                       # plugin Expo (nom Android)
│   ├── store/ios · store/android      # visuels des stores
│   ├── __tests__/ · app.json · eas.json · google-services.json
├── admin-console/                   # Console d'administration (React + Vite)
├── site/                            # Site public statique
├── ansible/                         # Rôles common, docker, app, web, security
├── .github/workflows/               # ci.yml, deploy.yml
├── docs/                            # mise en prod, CI/CD, backend, notifications…
├── tools/                           # ef-migrations.sh, scripts
├── src/ · tests/                      # prototype Avalonia (historique)
└── Directory.Packages.props · global.json · BeninConnect.sln
```

Structure d'un module backend

```
Modules/Messaging/
├── Contracts/        # seule partie visible des autres modules : DTO, interfaces, événements
├── Domain/           # entités et règles : Conversation, Message, MessageAttachment
├── Application/      # commandes, requêtes, gestionnaires, abstractions (IMessagingDb, IChatNotifier)
├── Infrastructure/   # EF Core (DbContext, migrations), SignalR (ChatHub), services
└── Api/              # MessagingModule : services et routes HTTP
```

5. Backend : monolithe modulaire

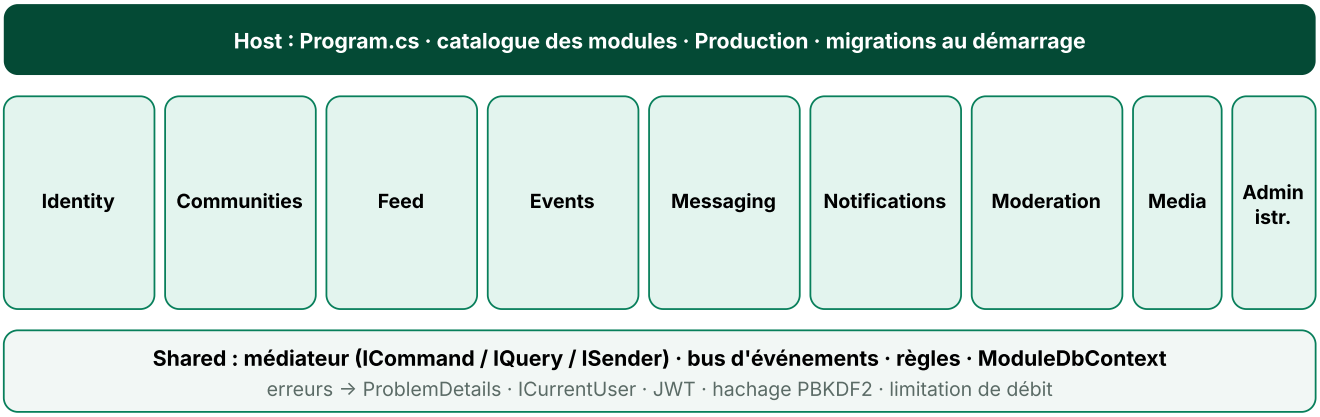


Figure 3 — Modules et noyau commun ; chaque module contient les couches Contracts, Domain, Application, Infrastructure, Api

Principes

- **Clean Architecture par module** : le domaine ne dépend de rien ; l'application dépend du domaine et d'abstractions ; l'infrastructure les implémente ; l'API ne fait que composer et exposer.
- **Frontières vérifiées par des tests** : `ArchitectureTests` analyse les sources et échoue si un module référence autre chose que les *Contracts* d'un autre module.
- **Une base logique par module** : chaque module a son `DbContext`, son schéma PostgreSQL (`identity`, `communities`, `feed`, `events`, `messaging`, `notifications`, `moderation`, `administration`) et sa table d'historique de migrations. Aucune clé étrangère entre modules : on référence un identifiant.
- **Médiateur maison** : commandes et requêtes (`ICommand<T>`, `IQuery<T>`), validation déclarative (`IValidatable`), gestionnaires enregistrés par espace de noms.
- **Erreurs normalisées** : `DomainException` → 400, `NotFoundException` → 404, `ForbiddenException` → 403, `ConflictException` → 409, toutes au format ProblemDetails.
- **Migrations** appliquées au démarrage (`Database:InitializeOnStartup`) ou par `dotnet BeninConnect.Api.dll --migrate`. Création : `tools/ef-migrations.sh <Nom> <Module>`.

Paquets principaux

Paquet	Version	Usage
Microsoft.EntityFrameworkCore / Npgsql.EntityFrameworkCore.PostgreSQL	10.0.0	Persistance, migrations
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.0	Jetons membres et administrateurs
Microsoft.AspNetCore.SignalR (framework)	10	Messagerie en temps réel
AWSSDK.S3	4.0.14	Stockage objet (Cloudflare R2, MinIO…)
Testcontainers.PostgreSql / Minio, xUnit, Mvc.Testing	4.4.0 / 2.9.2	Tests d'intégration

Les versions sont centralisées dans `Directory.Packages.props` (gestion centrale des paquets NuGet).

6. Modules métier et API

Module	Responsabilité	Routes principales	Migrations
Identity	Comptes, vérification email, mots de passe, profils, blocages entre membres, suspension, suppression de compte	`/api/auth/{login, register, verify-email, resend-code, forgot-password, reset-password, me}` · `/api/users (recherche, profil, PUT /me, me/blocked, {id}/block, POST /me/delete)` · `/api/admin/members`	InitialCreate, AddUserBlocks
Communities	Communautés publiques/privées, adhésions, demandes	`/api/communities (liste, détail, membres, join, membership, join-request)` · `/api/admin/communities`	InitialCreate
Feed	Publications (photo ou vidéo), j'aime, partages, commentaires, réactions, modération	`/api/feed` · `/api/posts (GET, POST, PUT, DELETE, like, share, comments, reactions)` · `/api/admin/posts (hide, unhide, delete)`	InitialCreate, AddPostEditedAt, AddPostVideo
Events	Événements, inscriptions, capacité, filtres	`/api/events (liste, catégories, villes, détail, participation)` · `/api/admin/events`	InitialCreate
Messaging	Conversations privées, messages, réactions, lecture, modification/suppression, pièces jointes, temps réel	`/api/conversations (direct, messages, attachments, PUT/DELETE message, read)` · `/api/messages/{id}/attachment` · `hub /hubs/chat`	InitialCreate, AddMessageEditDelete, AddMessageAttachments
Notifications	Centre de notifications, appareils push, envoi Expo	`/api/notifications (liste, unread-count, read, read-all, devices, devices/unregister)`	InitialCreate, AddPushDevices
Moderation	Signalements des membres, traitement par la modération	`POST /api/reports` · `/api/admin/reports (liste, resolve)`	InitialCreate
Media	Photos et vidéos publiques, pièces jointes privées, stockage local ou S3	`POST /api/media` · `POST /api/media/video` · `GET|HEAD /uploads/{nom}`	(aucune table)
Administration	Comptes administrateurs, permissions, statistiques, exports, journal d'audit, système	`/api/admin/{auth, stats, exports, admins, audit, system}`	InitialCreate

> Toutes les routes membres exigent un jeton (sauf inscription et connexion). Les routes `/api/admin/*` utilisent un schéma d'authentification distinct et des permissions par politique (`admin:{Permission}`). L'endpoint de réponse simulée (`demo-reply`) et la réinitialisation de démo ne sont exposés qu'en développement.

7. Communication entre modules

Les modules ne s'appellent jamais directement. Deux mécanismes, tous deux définis dans les *Contracts* :

- **Interfaces de module** (lecture synchrone) : `IIdentityModule` (noms, existence, suspension), `ICommunitiesModule` (adhésions), `IFeedModule`, `IMediaModule` (enregistrer/ouvrir une pièce jointe), `IModerationModule` …
- **Événements d'intégration** (bus en mémoire, `IEventBus`) : un module publie un fait, les autres réagissent.

Événement	Émetteur	Réactions
`MemberVerified`	Identity	Notifications : message de bienvenue
`MemberDeleted`	Identity	Feed, Communities, Events, Messaging, Notifications, Moderation : effacement de toutes les données du membre
`PostLiked` · `PostCommented` · `CommentReacted`	Feed	Notifications : notification + push à l'auteur
`CommunityJoined`	Communities	Notifications
`CommunityDeleted`	Communities	Feed et Events : publications et événements détachés de la communauté
`MessageSent`	Messaging	Notifications : notification + push aux destinataires
`ImagesReleased`	Identity, Feed, Communities, Events	Media : effacement des photos/vidéos devenues inutiles
`AttachmentsReleased`	Messaging	Media : effacement des pièces jointes privées
`NotificationRequested`	tout module	Notifications : notification générique

Exemple : suppression d'un compte

1. `POST /api/users/me/delete` avec le mot de passe ; Identity vérifie le mot de passe et supprime l'utilisateur.
2. Identity publie `MemberDeleted` puis `ImagesReleased` (photo de profil et couverture).
3. Chaque module efface ses données : publications, commentaires et j'aime ; communautés et événements créés, adhésions et inscriptions ; conversations et messages ; notifications et appareils ; signalements envoyés.
4. Media efface les fichiers publiés (`uploads/`) et les pièces jointes privées (`private/`).

8. Sécurité

Sujet	Mise en œuvre
Authentification	JWT signés (clé ≥ 32 caractères, `JWT_SIGNING_KEY`) ; schéma membre par défaut et schéma administrateur distinct. Un membre suspendu voit ses jetons refusés immédiatement (vérification à chaque requête).
Mots de passe	Hachage PBKDF2 ; codes de vérification et de réinitialisation à durée limitée et à tentatives limitées.
Autorisations	Contrôles métier dans les gestionnaires : seul l'auteur modifie ou supprime sa publication, seul l'expéditeur modifie (15 min) ou supprime son message, seuls les participants lisent une conversation et ses pièces jointes (404 sinon).
Limitation de débit	Politiques `Auth` (connexion, inscription, codes, suppression de compte) et `Uploads` (fichiers) en production.
Garde-fous de production	`Production.Validate` refuse de démarrer avec une configuration dangereuse : emails en mode journal, base sans SSL (sauf `DB_ALLOW_INSECURE` pour une base locale), clé JWT faible, S3 incomplet…
Fichiers	Format reconnu sur le contenu (signature binaire), jamais sur le nom ; noms aléatoires de 128 bits ; tailles maximales (photo 10 Mo, pièce jointe 20 Mo, vidéo 60 Mo). Pièces jointes dans un stockage privé jamais servi directement.
Jeton dans l'URL	Accepté uniquement pour `/hubs` (WebSocket) et `/api/messages/{id}/attachment` (image, audio, PDF ouverts par le téléphone). Caddy masque `access_token` dans ses journaux.
Transport	HTTPS partout (Caddy, Let's Encrypt), HSTS, en-têtes de sécurité, API liée à `127.0.0.1` ; pare-feu UFW (22, 80, 443) géré par Ansible.
Secrets	`.env` déposé à la main sur le serveur (droits 600), jamais dans git ; secrets GitHub (SSH, registre) ; identifiants Apple/Google gérés par EAS.
Contenu des membres	Acceptation des conditions d'utilisation (tolérance zéro) à l'inscription, signalement et blocage dans l'app, modération dans la console, suppression du compte dans l'app.

9. Temps réel et notifications

SignalR — `/hubs/chat`

Le hub est en lecture seule pour le client : le serveur pousse, le client n'appelle aucune méthode. L'utilisateur est identifié par la revendication `sub` du jeton (`MemberIdProvider`) ; les messages sont envoyés à tous les participants connectés, sur tous leurs appareils.

Événement	Déclencheur
`messageCreated(message)`	Nouveau message (texte ou pièce jointe)
`messageUpdated(message)`	Modification, suppression ou réaction
`conversationRead(conversationId, readerId)`	Le destinataire a lu les messages

Côté application (`src/features/realtime.ts`) : connexion WebSocket ouverte tant que le membre est connecté, reconnexion automatique, rattrapage des messages au retour au premier plan, mise à jour directe des caches TanStack Query. Une seule instance d'API : aucun fond de panier (Redis) n'est nécessaire.

Notifications push

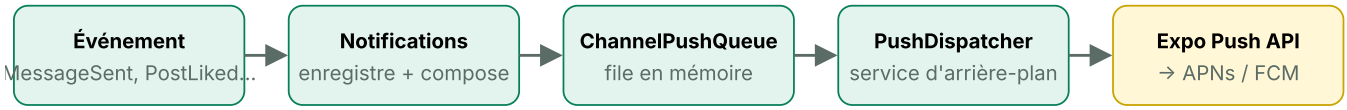


Figure 4 — Chaîne d'envoi des notifications push

- Enregistrement de l'appareil : `POST /api/notifications/devices` (jeton `ExponentPushToken[…]` , plateforme, catégories désactivées). Désinscription à la déconnexion.
- Envoi par lots de 100, badge = nombre de notifications non lues, données de navigation (`target` , `targetId`) ; les jetons invalides (`DeviceNotRegistered`) sont supprimés.
- iOS : clé APNs gérée par EAS. Android : projet Firebase, `google-services.json` et clé FCM V1 chargée dans EAS.
- Option : `EXPO_ACCESS_TOKEN` pour sécuriser l'envoi ; `Push:Provider=log` en test.

10. Médias et stockage

Zone	Contenu	Accès
Public `uploads/`	Photos de profil, couvertures, photos et vidéos des publications, miniatures	`https://api.monbenin.com/uploads/<nom>` ; en S3 avec domaine public : redirection 301 vers l'URL R2 (CDN)
Privé `private/`	Pièces jointes des discussions : photos, PDF, messages vocaux	Uniquement `GET /api/messages/{id}/attachment`, après vérification que le demandeur participe à la conversation

Formats et limites

Type	Formats acceptés (détection sur le contenu)	Taille max	Endpoint
Photo	JPEG, PNG, WebP, HEIC	10 Mo	`POST /api/media`
Vidéo	MP4, MOV (60 s, H.264 720p sur iPhone)	60 Mo	`POST /api/media/video`
Pièce jointe	Photos, PDF, audio (M4A, MP3, AAC, WAV, OGG, CAF)	20 Mo	`POST /api/conversations/{id}/attachments`

Fournisseurs

- **Local** (`MEDIA_PROVIDER=local`) : volume Docker `media` (`uploads/` et `private/`), servi par l'API (fichiers statiques, requêtes partielles natives), sauvegardé chaque nuit.
- **S3** (`MEDIA_PROVIDER=s3`), **en service** : Cloudflare R2, bucket `beninconnect-media` (photos et vidéos, URL publique `pub-….r2.dev`) et bucket `beninconnect-private` (pièces jointes, jamais public). Sans URL publique, l'API relaie les fichiers avec gestion des requêtes `Range`, indispensable au lecteur vidéo d'iOS.
- **Migration** : `server/tools/media-to-s3.sh` copie le volume vers les buckets (sans suppression, relançable). Les chemins en base (`uploads/<nom>`) ne changent pas.
- **Nettoyage** : un fichier remplacé ou dont le propriétaire est supprimé est effacé du stockage (événements `ImagesReleased` et `AttachmentsReleased`).

> L'URL `r2.dev` est prévue par Cloudflare pour le développement (débit limité, sans cache). Évolution conseillée : déplacer le DNS de `monbenin.com` chez Cloudflare et brancher le domaine `media.monbenin.com` sur le bucket public.

11. Application mobile

Pile technique

- Expo SDK 58, React Native 0.88, React 19.3, TypeScript
- Navigation : expo-router (groupes `(auth)` et `(app)`, onglets)
- État serveur : TanStack Query ; état local : Zustand
- Temps réel : @microsoft/signalr
- Médias : expo-image, expo-image-picker, expo-video, expo-video-thumbnails, expo-audio, expo-document-picker, expo-web-browser
- Notifications : expo-notifications, expo-device
- Stockage : expo-secure-store (jeton), AsyncStorage (préférences)

Identité de l'application

Nom affiché	BéninConnect
Identifiant	`com.hbatech.beninconnect.rn`
Version	1.0.0 (numéros de build gérés par EAS)
Appareils	iPhone (iPad désactivé), Android
Projet EAS	`@berihectors-team/beninconnect`
API	`https://api.monbenin.com`

Organisation du code

Dossier	Contenu
`src/app/`	Écrans : connexion, inscription, vérification, accueil, communautés, publication (création/modification), messages, conversation, événements, profil, paramètres, utilisateurs bloqués, suppression de compte
`src/data/`	`repositories.ts` (contrat `DataSource`), `api/apiDataSource.ts` (client HTTP, envoi XHR des fichiers), `mock/` (démonstration hors ligne), `types.ts`
`src/features/`	`queries.ts` (requêtes et mutations, mises à jour des caches), `realtime.ts`, `report.ts`, `block.ts`, `voice.ts`
`src/core/`	`config.ts` (URL de production, `DEV_TOOLS`), session, push, photos/vidéos, images, thème, préférences
`src/components/`	Composants d'interface, cartes (publication, message), pièces jointes, lecteur vidéo, réactions

Comportements notables

- **Production verrouillée** : hors développement (`__DEV__`) ou sans `EXPO_PUBLIC_DEV_TOOLS=1` , l'application utilise toujours l'API de production et masque les réglages de source de données.
- **Envoi de fichiers** par `XMLHttpRequest` (le `fetch` d'Expo 58 ne lit pas les fichiers `{ uri }`), avec progression pour la vidéo.
- **Messages vocaux** : enregistrement AAC (.m4a), lecture en mode silencieux sur le haut-parleur.
- **Notifications** : pas de bannière pour la discussion affichée ; toucher une notification ouvre l'écran sans empiler les discussions.
- **Autorisations iOS** : photos, appareil photo, micro (messages vocaux), notifications ; textes en français dans `app.json` .
- **Android** : icône adaptative dédiée, nom avec accent via `plugins/withAndroidAppName.js` , FCM via `google-services.json` .

12. Console d'administration et site public

Console d'administration (`admin-console/`)

Application React 19 + Vite + TanStack Query, authentifiée par le schéma administrateur de l'API, avec permissions par rôle.

Page	Rôle
Tableau de bord	Statistiques (membres, contenus, activité, pays)
Signalements	File de modération : traiter, masquer, supprimer, suspendre
Membres	Recherche, fiche, suspension, réactivation
Publications, Communautés, Événements	Consultation et modération des contenus
Administrateurs, Compte	Gestion des comptes et des permissions, mot de passe
Journal d'audit	Traçabilité des actions de modération
Exports, Système, Erreurs	Exports CSV, état du serveur et du stockage

> Le conteneur de la console a été retiré du déploiement de production (Traefik remplacé par Caddy). Elle peut être servie en statique par Caddy (par ex. `admin.monbenin.com`) le moment venu.

Site public (`site/`)

Page	Usage
`/`	Présentation, fonctionnalités, captures
`/support/`	Contact et questions fréquentes (URL de support des stores)
`/confidentialite/`	Politique de confidentialité (RGPD)
`/conditions/`	Conditions d'utilisation, tolérance zéro, modération
`/mentions-legales/`	Éditeur, hébergeur, propriété intellectuelle
`/supprimer-compte/`	Procédure de suppression (URL exigée par Google Play)

Pages statiques responsive (mode sombre automatique), 404 personnalisée, `robots.txt` , `sitemap.xml` . Déployées par la CI/CD dans `/var/www/monbenin` .

13. Configuration

Variables du serveur (`/opt/beninconnect/server/.env`)

Modèle : `server/.env.example`. Le fichier réel n'est jamais versionné ; Ansible vérifie sa présence et ses droits (600).

Variable	Rôle
`API_PORT`	Port local de l'API (8080), lié à 127.0.0.1
`DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD`	PostgreSQL (hôte : `host.docker.internal`, base native du VPS)
`DB_SSLMODE, DB_ALLOW_INSECURE, DB_POOL_SIZE`	SSL de la base, exception pour une base locale, taille du pool
`JWT_SIGNING_KEY`	Clé de signature des jetons (`openssl rand -base64 48`)
`EMAIL_PROVIDER, RESEND_API_KEY, EMAIL_FROM, EMAIL_FROM_NAME`	Emails via Resend (ou SMTP_*)
`EXPO_ACCESS_TOKEN`	Facultatif : sécurisation de l'envoi des push
`MEDIA_PROVIDER`	`local` ou `s3` (actuellement `s3`)
`S3_SERVICE_URL, S3_REGION, S3_BUCKET, S3_PRIVATE_BUCKET`	Cloudflare R2 : point d'accès, région `auto`, buckets public et privé
`S3_ACCESS_KEY, S3_SECRET_KEY`	Clé R2 limitée aux deux buckets (lecture/écriture)
`S3_PUBLIC_BASE_URL, S3_CREATE_BUCKET`	URL publique du bucket média ; création automatique (MinIO uniquement)
`CORS_ORIGIN_1, CORS_ORIGIN_2`	Origines autorisées (console web)
`ADMIN_BOOTSTRAP_*`	Premier administrateur créé au démarrage

Application mobile

Fichier	Contenu
`app.json`	Nom, identifiants, icônes, autorisations, plugins (image-picker, notifications, audio, splash, nom Android), projet EAS
`eas.json`	Profils `preview` et `production` (incrément automatique, .aab Android) ; envoi App Store (`ascAppId`) et Google Play (test interne, brouillon)
`google-services.json`	Projet Firebase (FCM) ; non secret
`.env.local`	`EXPO_PUBLIC_API_URL` pour pointer vers une API locale en développement

14. Infrastructure et déploiement

Élément	Détail
Serveur	VPS OVH, Ubuntu, IP 51.255.40.214, utilisateur `ubuntu`, dossier `/opt/beninconnect`
Domaines	`api.monbenin.com` (API), `monbenin.com` et `www` (site) ; DNS chez OVH ; emails OVH (MX) et Resend (DKIM)
Caddy	Installé sur l'hôte, configuration `/etc/caddy/sites/beninconnect.caddy` (importée par `/etc/caddy/Caddyfile`) ; HTTPS automatique, HSTS, corps de requête jusqu'à 70 Mo, journaux JSON avec masquage du jeton
Conteneurs (`docker-compose.prod.yml`)	`api` (image GHCR `beninconnect-api:sha-…`, 127.0.0.1:8080, volume `media`), `backup` (sauvegarde PostgreSQL quotidienne), `media-backup` (archive du volume média)
PostgreSQL	Installé nativement sur le VPS ; accès des conteneurs via `host.docker.internal` (règle UFW 172.16.0.0/12, `listen_addresses` et `pg_hba.conf` adaptés)
Stockage objet	Cloudflare R2 (photos, vidéos, pièces jointes)

Ansible (`ansible/`)

Rôle	Tâches
`common`	Paquets, fuseau horaire, swap, utilisateur de déploiement et clés SSH
`docker`	Installation de Docker et du plugin compose
`app`	Fichier compose, vérification du `.env`, connexion au registre GHCR, déploiement de l'image, contrôle de santé (journaux en cas d'échec)
`web`	Caddy : dossiers, import des sites, configuration validée par `caddy validate` puis rechargement ; synchronisation du site public
`security`	Pare-feu UFW (SSH, 80, 443), durcissement SSH

`site.yml` installe un serveur complet ; `deploy.yml` (rôles `app` et `web`) est exécuté à chaque déploiement.

15. CI/CD

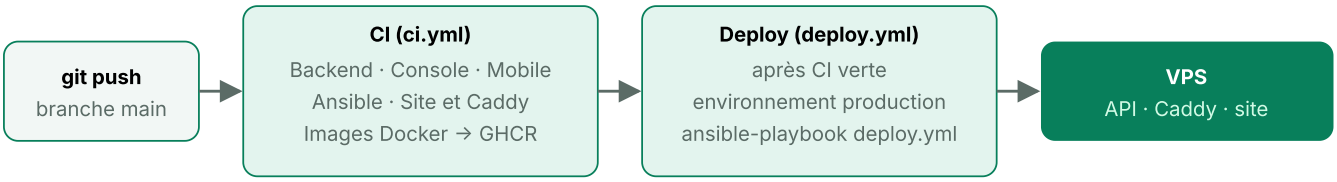


Figure 5 — Chaîne de livraison continue

Job	Vérifications
Backend (.NET 10)	Build Release, tests d'intégration (PostgreSQL et MinIO éphémères via Testcontainers), architecture modulaire, migrations à jour
Console admin	oxlint, vitest, `tsc`, build Vite
Application mobile	`tsc --noEmit`, Jest
Ansible	Vérification syntaxique des playbooks, ansible-lint
Site public et Caddy	`caddy validate` du Caddyfile, contrôle des liens internes du site
Images (main)	Image `ghcr.io/<propriétaire>/beninconnect-api`, étiquettes `sha-<commit>` et `latest`

Secrets GitHub (environnement `production`)

- `SSH_PRIVATE_KEY` : clé de déploiement (clé publique autorisée sur le VPS)
- `SSH_KNOWN_HOSTS` : empreinte du serveur (`ssh-keyscan`)
- `GHR_TOKEN` : lecture des images sur GHCR par le serveur

Déploiement manuel d'une version précise : Actions › Deploy › Run workflow avec une étiquette (`sha-1a2b3c4`), ce qui permet aussi de revenir en arrière.

16. Tests et qualité

Suite	Couverture
Backend — 67 tests xUnit	Architecture frontières entre modules, couches Domaine règles pures (j'aime, adhésion, fenêtre de modification des messages...) Intégration API réelle sur PostgreSQL : authentification, contrats, administration, push, emails, suppression de compte, blocage, temps réel, pièces jointes, vidéo S3 stockage MinIO : photos, vidéo avec requêtes partielles, pièces jointes privées (en CI) Production garde-fous de configuration
Mobile — 46 tests Jest	Source de démonstration et client API : photos, suppression, signalement, blocage, modifications, pièces jointes, vidéo, formats, règles, réactions
Console — vitest	Utilitaires (format, permissions, client API)

Exécution locale

```
# Backend (Docker requis pour Testcontainers, ou BC_TEST_POSTGRES=« Host=…;Username=…;Password=… »)
dotnet test server/tests/BeninConnect.Api.Tests

# Mobile
cd mobile-expo && npx tsc --noEmit && npx jest

# Nouvelle migration pour un module
tools/ef-migrations.sh AddQuelqueChose Feed
```

17. Publication sur les stores

Commandes

```
cd mobile-expo
eas build -p all --profile production     # builds iOS (.ipa) et Android (.aab) sur les serveurs EAS
eas submit -p ios --latest                # App Store Connect (ascAppId 6832112483)
eas submit -p android --latest            # Google Play, test interne (google-play-service-account.json)
```

App Store

- Fiche en français : nom, sous-titre, description, mots-clés, texte promotionnel
- Captures iPhone 1206 × 2622 et bannière d'en-tête (`store/ios/`)
- URL de support et de confidentialité sur `monbenin.com`
- App Privacy : coordonnées, contenus, photos/vidéos, audio, identifiants
- DAC7, copyright, compte de test pour la relecture
- Clé APNs (push) gérée par EAS

Google Play

- Captures 1080 × 1920, icône 512, bannière 1024 × 500 (`store/android/`)
- API Google Play Developer activée, compte de service invité
- Premier envoi manuel du .aab, puis EAS
- Sécurité des données, classification, public cible
- URL de suppression de compte : `/supprimer-compte/`
- Comptes personnels récents : test fermé (12 testeurs, 14 jours) avant la production

18. Points restants et évolutions

Priorité	Sujet	Détail
Haute	Revue App Store	Répondre à la demande d'informations (règle 2.1) avec la build incluant le blocage, la vidéo et les notes de relecture
Haute	Domaine média	DNS de `monbenin.com` chez Cloudflare, puis `media.monbenin.com` sur le bucket R2 (remplace l'URL `r2.dev` limitée)
Moyenne	Sauvegardes hors site	Copier les sauvegardes PostgreSQL du VPS vers R2 ou un autre stockage
Moyenne	Mentions légales	Ajouter l'adresse du siège et le numéro RCCM de HBA Tech et Trade
Moyenne	Console d'administration	La publier (statique derrière Caddy, ex. `admin.monbenin.com`)
Moyenne	Vidéo sur Android	Compression avant envoi (pas de conversion automatique comme sur iPhone)
Basse	Dépendances	Mettre à jour les paquets signalés vulnérables (Microsoft.OpenApi, System.Security.Cryptography.Pkcs, SSH.NET côté tests)
Basse	Langues	Anglais dans l'application et sur le site
Basse	Montée en charge	Plusieurs instances de l'API : fond de panier SignalR (Redis) et file de push partagée

Documentation du dépôt

Fichier	Sujet
`docs/MISE_EN_PROD.md`	Base de données, médias et S3, emails, migrations, Caddy
`docs/BACKEND_MODULAIRE.md`	Règles du monolithe modulaire
`docs/CI_CD.md`	Workflows, secrets, site public, retour arrière
`docs/NOTIFICATIONS_PUSH.md`	APNs, FCM, Expo
`ansible/README.md`	Installation et déploiement du serveur

Annexe A. Référence complète de l'API

Base : `https://api.monbenin.com`. Corps et réponses en JSON (camelCase, énumérations en texte). Authentification : en-tête `Authorization: Bearer <jeton>`. Erreurs : `400` règle métier ou validation (`{ "title", "errors" }`), `401` non connecté, `403` interdit, `404` introuvable ou non autorisé, `409` conflit, `429` trop de requêtes.

Authentification et membres — Identity		
POST	`/api/auth/register`	Inscription (code de vérification envoyé) — public, limité
POST	`/api/auth/verify-email`	Vérifie le code, retourne jeton + profil — public
POST	`/api/auth/resend-code`	Renvoie le code de vérification — public
POST	`/api/auth/login`	Connexion, retourne jeton + profil — public
POST	`/api/auth/forgot-password`	Envoie un code de réinitialisation — public
POST	`/api/auth/reset-password`	Nouveau mot de passe avec le code — public
GET	`/api/auth/me`	Profil du membre connecté
GET	`/api/users?search=`	Recherche de membres
GET	`/api/users/interests`	Catalogue des centres d'intérêt
POST	`/api/users/batch`	Profils par identifiants (affichage des auteurs)
GET	`/api/users/{id}`	Profil d'un membre
PUT	`/api/users/me`	Modifier son profil (photo, couverture comprises)
GET	`/api/users/me/blocked`	Membres que j'ai bloqués
POST	`/api/users/{id}/block`	Bloquer un membre (idempotent, 204)
DELETE	`/api/users/{id}/block`	Débloquer (204)
POST	`/api/users/me/delete`	Supprimer son compte (mot de passe), 204
Communautés — Communities		
GET	`/api/communities`	Liste (recherche, catégorie)
GET	`/api/communities/categories`	Catégories
GET	`/api/communities/join-requests/mine`	Mes demandes d'adhésion en attente
POST	`/api/communities/batch`	Communautés par identifiants
GET	`/api/communities/{id}`	Détail
GET	`/api/communities/{id}/members`	Membres
POST	`/api/communities`	Créer une communauté
POST	`/api/communities/{id}/join`	Rejoindre (ou demander si privée)
DELETE	`/api/communities/{id}/membership`	Quitter
DELETE	`/api/communities/{id}/join-request`	Annuler sa demande
GET	`/api/users/{id}/communities`	Communautés d'un membre
Fil d'actualité — Feed		
GET	`/api/feed?skip=&take=`	Fil paginé (50 au plus par page)
GET	`/api/posts?authorId=&communityId=&search=`	Publications filtrées
GET	`/api/posts/{id}`	Détail (404 si invisible ou auteur bloqué)
POST	`/api/posts`	Publier (texte, imagePath ou videoPath, visibilité, communauté)

PUT	`/api/posts/{id}`	Modifier (auteur)
DELETE	`/api/posts/{id}`	Supprimer (auteur, 204)
POST	`/api/posts/{id}/like`	J'aime / je n'aime plus
POST	`/api/posts/{id}/share`	Compte un partage
GET	`/api/posts/{id}/comments`	Commentaires (sans ceux des membres bloqués)
POST	`/api/posts/{id}/comments`	Commenter (201)
POST	`/api/posts/{id}/comments/{cid}/reactions`	Réagir à un commentaire (emoji)

Événements — Events		
GET	`/api/events`	Liste (recherche, catégorie, ville, période)
GET	`/api/events/categories`	Catégories
GET	`/api/events/cities`	Villes ayant des événements
GET	`/api/events/{id}`	Détail
POST	`/api/events`	Créer un événement
POST	`/api/events/{id}/participation`	Participer / se désinscrire
GET	`/api/communities/{id}/events`	Événements d'une communauté
GET	`/api/users/{id}/events`	Événements d'un membre
Messagerie — Messaging		
GET	`/api/conversations`	Mes discussions (dernier message, non-lus, isBlocked)
GET	`/api/conversations/unread-count`	Nombre de messages non lus
POST	`/api/conversations/direct`	Ouvrir (ou retrouver) une discussion avec un membre
GET	`/api/conversations/{id}`	Résumé d'une discussion
GET	`/api/conversations/{id}/messages`	Messages
POST	`/api/conversations/{id}/messages`	Envoyer un message texte (refusé si blocage)
POST	`/api/conversations/{id}/attachments`	Envoyer une pièce jointe (multipart : file, caption, durationMs)
PUT	`/api/conversations/{id}/messages/{mid}`	Modifier (expéditeur, 15 min)
DELETE	`/api/conversations/{id}/messages/{mid}`	Supprimer (expéditeur)
POST	`/api/conversations/{id}/messages/{mid}/reactions`	Réagir à un message
POST	`/api/conversations/{id}/read`	Marquer comme lu
GET	`/api/messages/{mid}/attachment`	Télécharger la pièce jointe (participants ; jeton accepté en `?access_token=` ; plages Range)
WS	`/hubs/chat`	SignalR : messageCreated, messageUpdated, conversationRead
Notifications, médias, modération		
GET	`/api/notifications`	Mes notifications
GET	`/api/notifications/unread-count`	Nombre de non-lues
POST	`/api/notifications/{id}/read`	Marquer comme lue
POST	`/api/notifications/read-all`	Tout marquer comme lu
POST	`/api/notifications/devices`	Enregistrer l'appareil push (jeton Expo, plateforme, catégories coupées)
POST	`/api/notifications/devices/unregister`	Désinscrire l'appareil
POST	`/api/media`	Envoyer une photo (multipart), retourne le chemin — limité
POST	`/api/media/video`	Envoyer une vidéo (multipart, 60 Mo) — limité
GET/HEAD	`/uploads/{nom}`	Fichier public (disque, ou redirection 301 vers R2)
POST	`/api/reports`	Signaler (targetType, targetId, reason, details)

Administration (jeton administrateur, permission indiquée)		
POST	`/api/admin/auth/login · GET /auth/me · POST /auth/password`	Connexion, profil, changement de mot de passe
GET	`/api/admin/stats`	Tableau de bord — DashboardView
GET	`/api/admin/exports/{kind}`	Exports CSV — ExportsDownload
GET/POST/PUT	`/api/admin/admins …`	Comptes administrateurs — AdminsManage
GET	`/api/admin/audit`	Journal d'audit — AuditView
GET/POST	`/api/admin/system · /system/reset-demo`	État du système — SystemView ; réinitialisation (dév.) — SystemReset
GET/POST	`/api/admin/members · /{id} · /suspend · /reactivate`	Membres — MembersView, MembersSuspend
GET/POST/DEL	`/api/admin/posts · /comments · /hide · /unhide`	Modération des contenus — ContentModerate
GET/POST	`/api/admin/reports · /{id}/resolve`	Signalements — ReportsManage
GET/DEL	`/api/admin/communities · /api/admin/events`	CommunitiesManage, EventsManage
GET	`/health`	Santé : chaque base de module joignable

Annexe B. Modèle de données

Une seule base PostgreSQL, un **schéma par module**. Aucune clé étrangère entre schémas : un module ne référence un autre que par identifiant (UUID) et passe par son contrat pour lire ses données. Dates en `timestamp with time zone` (UTC).

Table	Colonnes
`identity.users`	Id, FirstName, LastName, Email (unique), PasswordHash, Profession, Country, City, OriginCity, Bio, Interests, AvatarColor, AvatarImage, CoverImage, JoinedAt, IsEmailVerified, VerificationCode, PasswordResetCode, PasswordResetExpiresAt, PasswordResetAttempts, IsSuspended, SuspendedAt, SuspensionReason
`identity.user_blocks`	BlockerId + BlockedId (clé), CreatedAt ; index sur BlockedId
`communities.communities`	Id, Name, Description, Category, Country, City, CoverImage, Visibility (Public, Private), OwnerId, CreatedAt
`communities.members`	CommunityId + UserId, Role (Member, Moderator, Owner), JoinedAt
`communities.join_requests`	Id, CommunityId, UserId, RequestedAt
`feed.posts`	Id, AuthorId, CommunityId, Content, ImagePath, VideoPath, Visibility (Public, CommunityMembers, OnlyMe), CreatedAt, EditedAt, CommentCount, ShareCount, IsHidden, HiddenReason
`feed.post_likes`	PostId + UserId, LikedAt
`feed.comments`	Id, PostId, AuthorId, Content, CreatedAt
`feed.comment_reactions`	CommentId + UserId, Emoji, ReactedAt
`events.events`	Id, Title, Description, Category, StartsAt, EndsAt, Address, City, Country, Capacity, ImagePath, OrganizerId, CommunityId
`events.participants`	EventId + UserId, RegisteredAt
`messaging.conversations`	Id, CreatedAt
`messaging.participants`	ConversationId + UserId
`messaging.messages`	Id, ConversationId, SenderId, Content, SentAt, EditedAt, IsRead, IsDeleted, IsSimulated ; pièce jointe (type possédé) : nom de stockage, type (Image, Audio, File), nom de fichier, type MIME, taille, durée
`messaging.message_reactions`	MessageId + UserId, Emoji, ReactedAt
`notifications.notifications`	Id, UserId, Category (Social, Communities, Messages, Events, System), Title, Body, Target (Post, Community, Conversation, Event, Profile), TargetId, IsRead, CreatedAt
`notifications.push_devices`	Id, UserId, Token (ExponentPushToken), Platform, RegisteredAt, LastSeenAt (+ catégories coupées)
`moderation.reports`	Id, ReporterId, TargetType (Post, Comment, User, Community), TargetId, Reason, Details, Status (Open, Resolved, Dismissed), Resolution, CreatedAt, ClosedAt, ClosedByAdminId
`administration.admin_users`	Id, Email, DisplayName, PasswordHash, Role, IsActive, CreatedAt, LastLoginAt
`administration.audit`	Id, AdminId, AdminName, Action, TargetType, TargetId, Details, At

Fichiers (stockage objet)

Emplacement	Contenu
`beninconnect-media / uploads/`	Photos de profil et de couverture, photos et vidéos des publications. Lecture publique via l'URL R2 ; noms aléatoires (32 caractères hexadécimaux + extension).
`beninconnect-private / private/`	Pièces jointes des discussions (photos, PDF, audio). Jamais publiques : servies par l'API aux seuls participants.

Les fichiers sont supprimés par les événements `ImagesReleased` et `AttachmentsReleased` lorsque la publication, le message ou le compte qui les référence disparaît.

Annexe C. Flux détaillés

C.1 Envoi d'un message avec temps réel et push

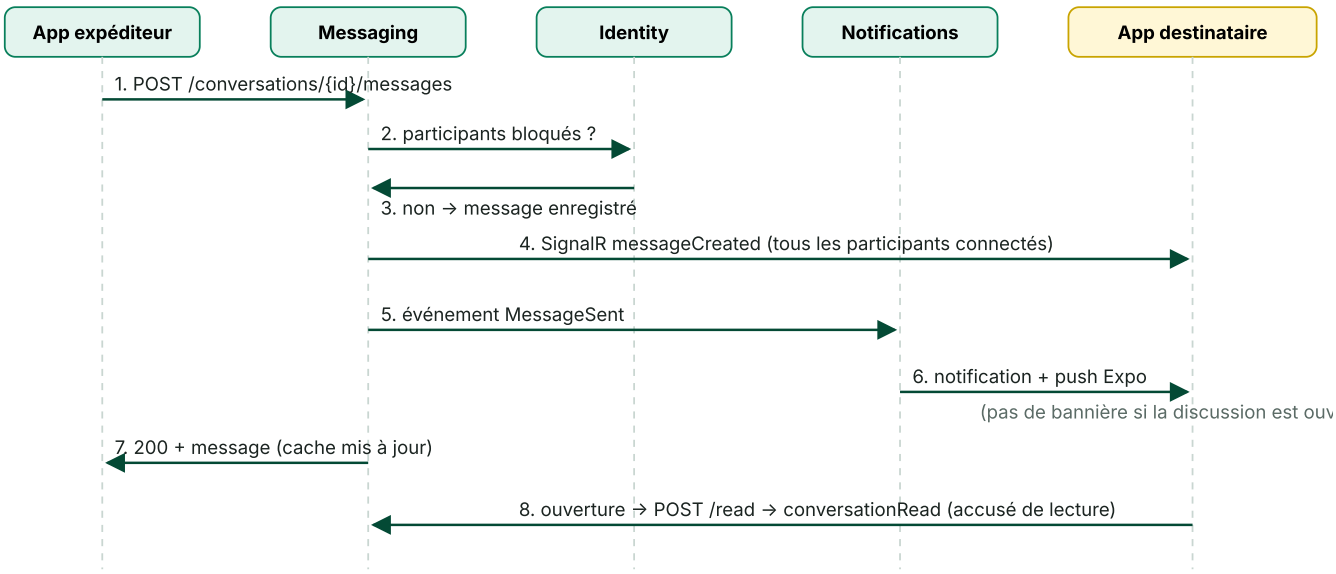


Figure 6 — Séquence d'envoi d'un message

C.2 Publication d'une vidéo

1. L'app choisit la vidéo (60 s au plus) ; sur iPhone, export H.264 1280×720 ; vignette générée localement ; contrôle des 60 Mo.
2. `POST /api/media/video` par XMLHttpRequest avec progression ; l'API vérifie la signature MP4/MOV, nomme le fichier et l'écrit dans R2 (`uploads/`).
3. `POST /api/posts` avec `videoPath` ; la publication apparaît dans le fil.
4. Lecture : expo-video lit directement l'URL publique R2 (requêtes partielles gérées par R2).

C.3 Lecture d'une pièce jointe (photo, PDF, message vocal)

1. L'app construit `/api/messages/{id}/attachment?access_token=…` (les lecteurs natifs ne savent pas envoyer d'en-tête).
2. L'API vérifie que le demandeur participe à la discussion, puis lit le fichier dans le bucket privé. Le flux S3 est chargé en mémoire (20 Mo au plus) pour accepter les **plages Range**, exigées par le lecteur audio d'iOS.
3. Avant la lecture d'un message vocal, l'app passe la session audio en mode lecture (`playsInSilentMode`) : le son sort sur le haut-parleur même avec le bouton silencieux.

C.4 Suppression d'un compte

1. `POST /api/users/me/delete` avec le mot de passe ; Identity supprime l'utilisateur et ses blocages.
2. Publication de `MemberDeleted` : Feed, Communities, Events, Messaging, Notifications et Moderation effacent les données du membre ; les communautés possédées sont supprimées (`CommunityDeleted`).
3. Publication de `ImagesReleased` et `AttachmentsReleased` : Media supprime les fichiers.
4. L'app désinscrit l'appareil push, efface le jeton et revient à l'écran de connexion.

C.5 Blocage

1. `POST /api/users/{id}/block` ; ligne `identity.user_blocks`.
2. Feed (publications, commentaires) et Messaging (envoi, ouverture de discussion, indicateur `isBlocked`) interrogent `IIdentityModule.GetBlockedRelationsAsync`, qui renvoie les membres bloqués *et* ceux qui ont bloqué le demandeur.
3. L'app invalide les caches (fil, publications, commentaires, discussions) et affiche le bandeau dans la discussion.

Annexe D. Exploitation au quotidien

Commandes utiles sur le serveur

```
# État et journaux de l'API
cd /opt/beninconnect/server
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f --tail 200 api
curl -s https://api.monbenin.com/health

# Journaux Caddy (accès, erreurs)
sudo tail -f /var/log/caddy/api.log
sudo journalctl -u caddy -n 100

# Redémarrer l'API (les migrations s'appliquent au démarrage)
docker compose -f docker-compose.prod.yml up -d api

# Sauvegardes (conteneurs backup et media-backup, chaque nuit)
ls -lh backups/daily/ backups/media/
# Restauration d'une sauvegarde PostgreSQL dans une base vide
gunzip -c backups/daily/<fichier>.sql.gz | psql "<chaîne de connexion>"
```

Déploiement et retour arrière

- **Normal** : un push sur `main` lance la CI ; si tout est vert, le workflow Deploy construit l'image, la publie sur GHCR et lance Ansible sur le VPS (API, Caddy, site).
- **Retour arrière** : relancer Deploy sur un commit précédent, ou remettre l'image précédente dans `API_IMAGE` (`.env`) puis `docker compose up -d api` . Les migrations ne sont pas annulées automatiquement : prévoir une migration corrective.
- **Application mobile** : une correction côté serveur prend effet immédiatement ; une correction côté application exige une nouvelle build EAS et une nouvelle revue.

Incidents connus et solutions

Symptôme	Cause	Solution
Envoi de photo : « Impossible de joindre le serveur »	Le `fetch` d'Expo 58 refuse les fichiers `{ uri }`	Envois par XMLHttpRequest (`xhrFetch.ts`), en place
Message vocal illisible sur iPhone	Pas de plages Range sur les pièces jointes R2 ; mode silencieux	Mise en mémoire + Range côté API ; mode audio de lecture côté app (corrigé)
Plusieurs écrans de discussion empilés après des notifications	Chaque toucher ajoutait un écran	Discussion ouverte suivie ; remplacement au lieu d'empilement (corrigé)
`curl -I` sur un média : 405	HEAD non géré	`MapMethods GET, HEAD` (corrigé)
« Notifications push indisponibles : Not Found »	Projet EAS ou identifiants push absents	`eas init` , clé APNs / FCM V1 dans EAS
Installation sur iPhone : APIInternalError	Version TestFlight déjà installée	Supprimer l'app puis relancer
L'API refuse de démarrer en production	Garde-fou `Production.Validate`	Lire le message : variable manquante ou dangereuse dans `.env`
Trop de requêtes (429)	Limitation de débit	Ajuster `RateLimits__*` si l'usage est légitime

Annexe E. Historique des évolutions récentes

Date	Évolution
6 nov. 2026	Schémas initiaux des huit modules avec base de données (`InitialCreate`), mise en production sur le VPS
7 nov. 2026	Appareils push (`AddPushDevices`), envoi des photos fiabilisé ; ensuite : suppression de publication et de compte, signalement, partage natif, site public déployé par la CI
8 nov. 2026	Messagerie en temps réel (SignalR), modification et suppression de messages et de publications (`AddPostEditedAt` , `AddMessageEditDelete`), pièces jointes (`AddMessageAttachments`), stockage Cloudflare R2, publications vidéo (`AddPostVideo`), notifications push Android (FCM)
8 nov. 2026	Lecture des messages vocaux corrigée (Range, mode silencieux) ; notifications sans empilement d'écrans
9 nov. 2026	Blocage d'utilisateurs (règle Apple 1.2) ; écran « Utilisateurs bloqués » ; conditions d'utilisation et confidentialité liées à l'inscription ; mentions « démonstration » retirées en production

Annexe F. Glossaire

Terme	Définition
Monolithe modulaire	Une seule application déployée, découpée en modules indépendants (données, contrats, événements séparés).
Contrat de module	Interface publique (`IIdentityModule` …) : seul point d'accès d'un module à un autre.
Événement d'intégration	Message publié par un module (`MemberDeleted` …) et traité par les autres, sans dépendance directe.
SignalR	Bibliothèque temps réel de .NET (WebSocket) utilisée pour la messagerie.
EAS	Expo Application Services : builds iOS/Android, identifiants de signature, envoi aux stores.
R2	Stockage objet de Cloudflare compatible S3, sans frais de sortie.
APNs / FCM	Services de notifications push d'Apple et de Google.
Plage Range	Requête HTTP d'une partie d'un fichier (lecture audio et vidéo progressive).
Caddy	Serveur web frontal : HTTPS automatique, relais vers l'API, site statique.
Ansible	Outil d'automatisation qui configure et déploie le serveur.